

back to all peers, and each one independently checks and commits it to its own ledger. This clean split between who authorises a transaction and who orders it is exactly what allows Fabric to scale smoothly without ever depending on a single point of failure.

The prerequisites for setting up this private blockchain network are:

- Windows 11 with WSL 2 enabled
- Docker Desktop 4.x or later — WSL 2 backend enabled
- A Linux distribution inside WSL 2 (Ubuntu or Debian recommended)
- curl and git (available by default in most Linux distros)
- ~2GB free for Docker image downloads

Setting up the environment

Step 1 — Enable WSL 2 and configure Docker Desktop:

Fabric's shell scripts require a Linux environment.

WSL 2 provides this on Windows. Open PowerShell as 'Administrator' and run the command below. Reboot when prompted.

```
wsl --install
```

Once rebooted, install Docker Desktop and open *Settings*. Under *General*, confirm that 'Use the WSL 2-based engine' is turned on. Then go to *Resources* > *WSL Integration*, toggle on your Linux distribution, and click *Apply & Restart*. Once done, open your Linux terminal and run `docker ps` — an empty container table with no errors confirms everything is working.

Step 2 — Download Hyperledger Fabric: Fabric's install script fetches the peer/orderer binaries, the sample chaincode, and all Docker images in one go:

```
cd ~
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
```

This takes 10–20 minutes depending on connection speed. When done, a `fabric-samples` directory appears in the home folder with the binaries inside it.

All the scripts and commands used in this article are available at <https://github.com/berlinflx/hyperledger-fabric-experiment>.

Spinning up the network

All commands from here run inside the `test-network` directory:

```
cd ~/fabric-samples/test-network
./network.sh down          # clear any leftover state
./network.sh up createChannel -c mychannel -ca
```

This generates cryptographic material for all identities, starts the eight containers, creates a channel named *mychannel*, and joins both peers to it. When finished, the terminal prints:

```
Channel 'mychannel' joined.
```

Deploying the smart contract

The chaincode used here is `asset-transfer-basic`. It stores assets on the ledger with five fields — ID, colour, size, owner, and appraised value, and supports create, read, update, delete, transfer, and list operations.

Deploy the JavaScript version (no local Node.js needed — Fabric builds it inside a container):

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

The process installs the package on both peers, collects approval from each organisation's admin, and commits the definition to *mychannel*. Successful output ends with:

```
Chaincode definition committed on channel 'mychannel'
```

```
(berlinflx@AkaXberlin)~/fabric-samples/test-network
$ docker ps
CONTAINER ID   IMAGE                                COMMAND
49b4bb47e007   hyperledger/fabric-peer:latest      "peer node start"
7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
11fd83331ca9   hyperledger/fabric-orderer:latest   "orderer"
7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp
d451a78579b5   hyperledger/fabric-peer:latest      "peer node start"
9051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp
4227beb75ad8   hyperledger/fabric-ca:latest         "sh -c 'fabric-ca-se..."
7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:17054->17054/tcp
7adc98b1e58d   hyperledger/fabric-ca:latest         "sh -c 'fabric-ca-se..."
9054/tcp, 0.0.0.0:19054->19054/tcp, [::]:19054->19054/tcp
0ecf8c9ae90b   hyperledger/fabric-ca:latest         "sh -c 'fabric-ca-se..."
8054/tcp, 0.0.0.0:18054->18054/tcp, [::]:18054->18054/tcp

(berlinflx@AkaXberlin)~/fabric-samples/test-network
$
```

Figure 1: All eight containers are up after `network.sh up createChannel`